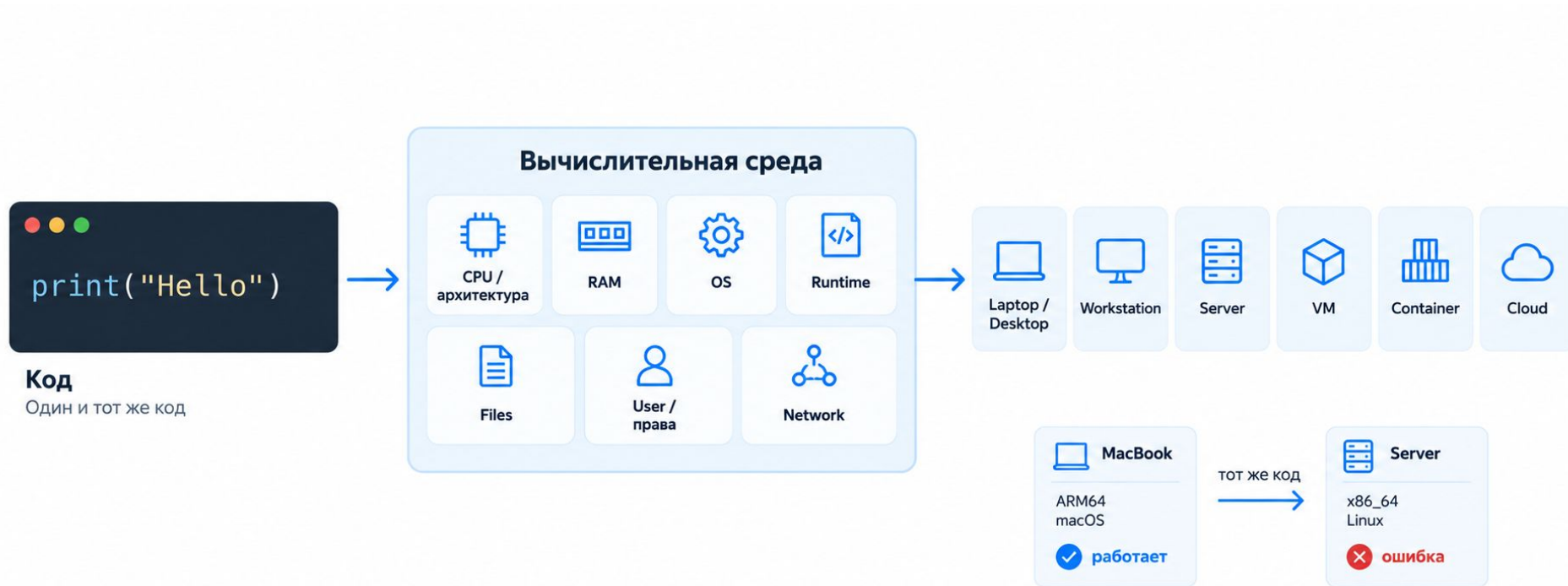


Автоматизация разработки

ПО лекция 3

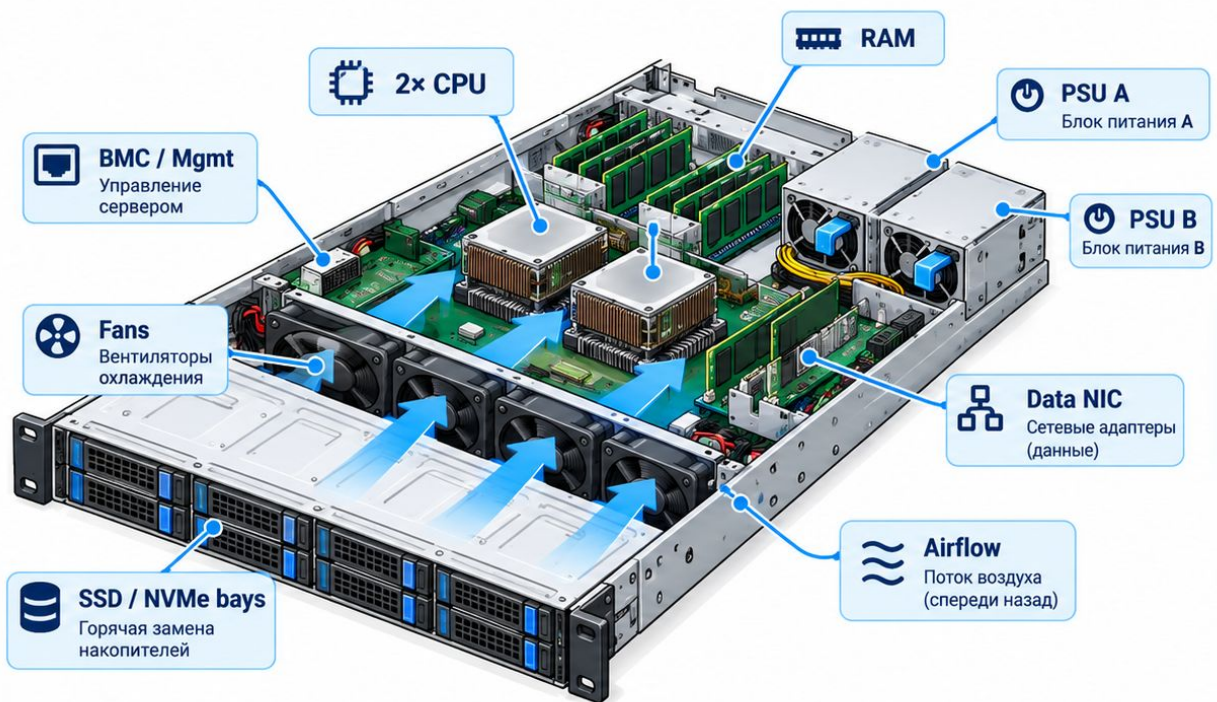
Вычислительная среда и воспроизводимость

Где выполняется ваш код?



Код один. Среда и место выполнения могут быть разными.

Сервер . . .



24/7
Надёжная работа



Hot-swap
Горячая замена



Redundant PSU
Резервирование питания



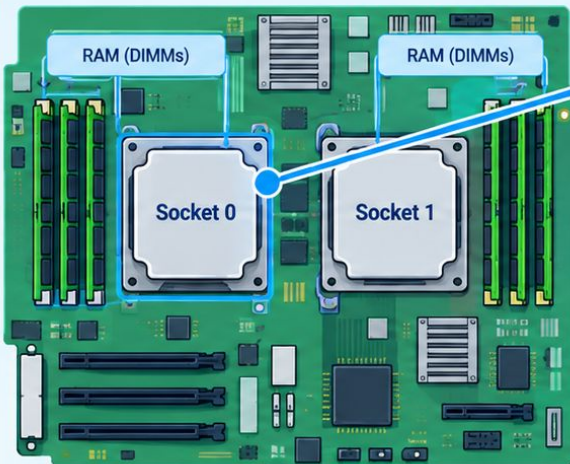
Remote management
Удалённое управление



Процессор . . .



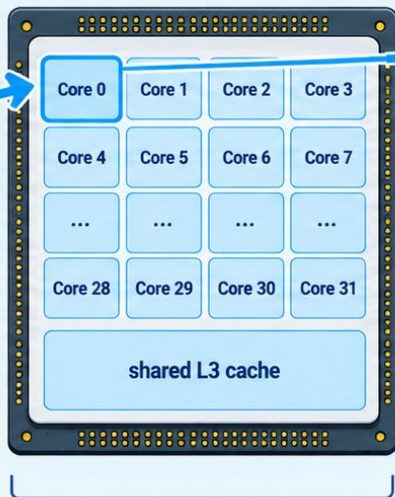
Server board
2 CPU sockets



2 CPU sockets



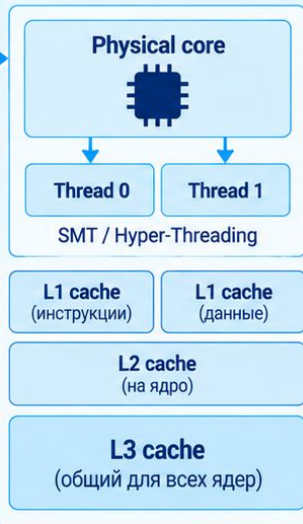
CPU package
32 cores (example)



32 cores



One core
1 physical core → 2 threads



Итоговая арифметика

**2 sockets ×
32 cores/socket ×
2 threads/core =**

128 logical CPUs



Реальный вывод (Linux)

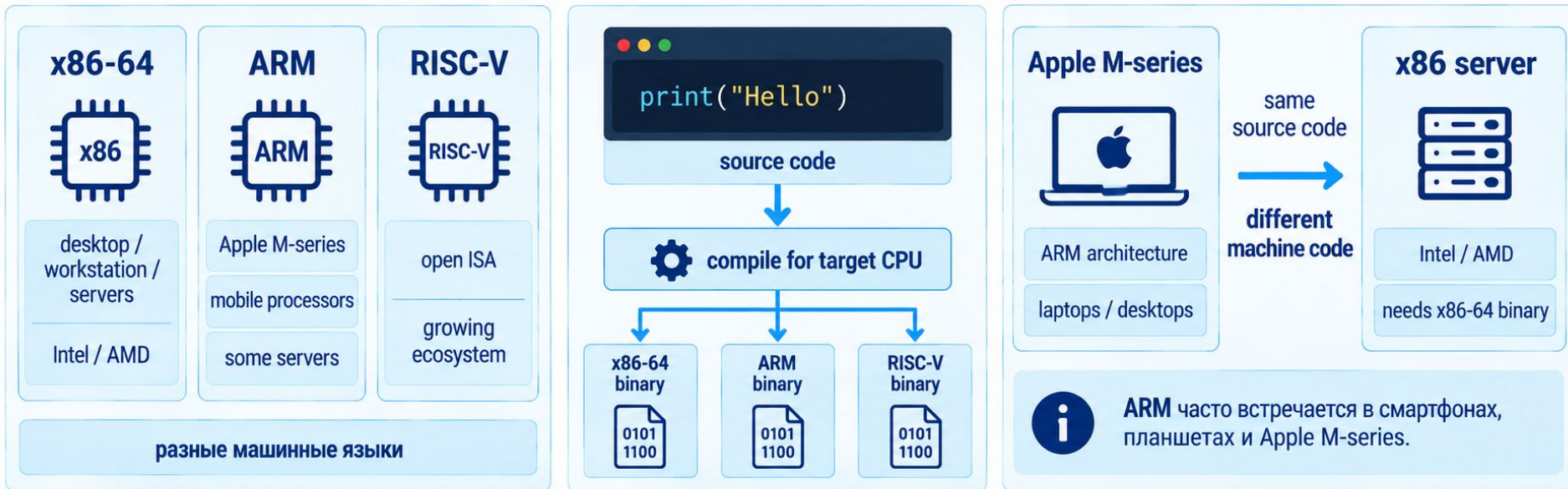
\$ lscpu

```
CPU(s):          128
Socket(s):        2
Core(s) per socket: 32
Thread(s) per core: 2
```



ОС видит logical CPUs,
а не только физические процессоры.

Процессоры . . .



Исходный код переносим, машинный код зависит от архитектуры CPU.

Оптимизации

Scalar / обычные вычисления

Сложение элементов массивов по одному:

a1 + b1 → c1

a2 + b2 → c2

a3 + b3 → c3

a4 + b4 → c4

одна инструкция обрабатывает один элемент

SIMD / векторные инструкции

Сложение сразу нескольких элементов с помощью векторных регистров:

$\begin{bmatrix} a1 & a2 & a3 & a4 \end{bmatrix} + \begin{bmatrix} b1 & b2 & b3 & b4 \end{bmatrix} \rightarrow \begin{bmatrix} c1 & c2 & c3 & c4 \end{bmatrix}$
вектор A вектор B вектор C

одна инструкция обрабатывает сразу несколько элементов

SSE 128 bit
4 × float32 

AVX2 256 bit
8 × float32 

AVX-512 512 bit
16 × float32 

Полезны для математики, мультимедиа, научных вычислений, машинного обучения, обработки данных.

Компилятор и сборка

Один и тот же исходный код:

```
void add(float *a, float *b, float *c, int n) {  
    for (int i = 0; i < n; i++) {  
        c[i] = a[i] + b[i];  
    }  
}
```



Компилятор
(gcc, clang и др.)

Generic build

```
gcc app.c
```

лучше
переносимость

Native build

```
gcc -march=native  
app.c
```

лучше оптимизация
под текущий CPU

Цена оптимизации / совместимость



Сервер сборки (x86-64)
с поддержкой **AVX2**



Собранный бинарный файл
содержит инструкции
AVX2



Старый x86-64 CPU
без поддержки **AVX2**

AVX2



Illegal instruction
(программа аварийно завершается)

Оба CPU — x86-64,
но набор доступных инструкций разный.

x86-64

одна архитектура
(семейство команд)



Intel

разные поколения,
разные сильные стороны



AMD

разные поколения,
разные сильные стороны

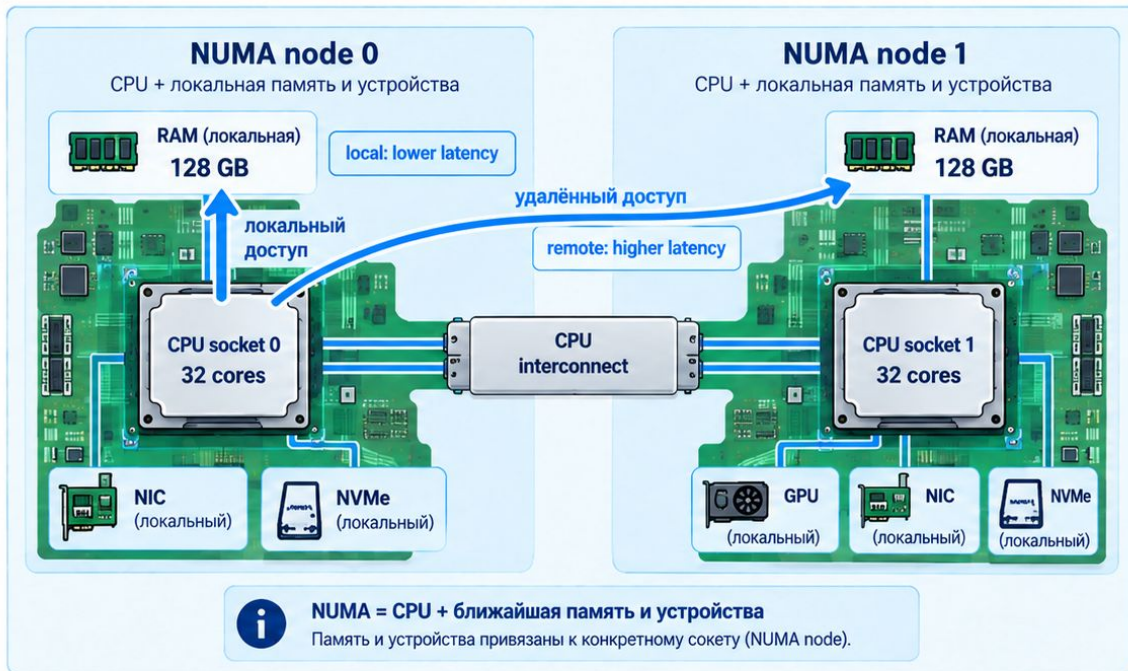


расширения ISA
влияют на совместимость
SSE, AVX2, AVX-512 и др.



микроархитектура
влияет на производительность
разные поколения CPU, разные оптимизации

Мифические NUMA ноды



Топология важна



Local ≠ Remote



Physical ≠ Allocated

RAM и не только

Memory levels (by closeness to CPU)



CPU cache
~ 1–10 ns
(nanoseconds)



Local RAM
~ 100 ns



Remote RAM
~ 100–200+ ns

RAM
(still memory)



Swap on NVMe / SSD
~ 10–100 µs
(microseconds)



Swap on HDD
~ 1–10 ms
(milliseconds)

Farther from CPU →
higher latency



Topology matters

Чем дальше от CPU — тем выше задержки.



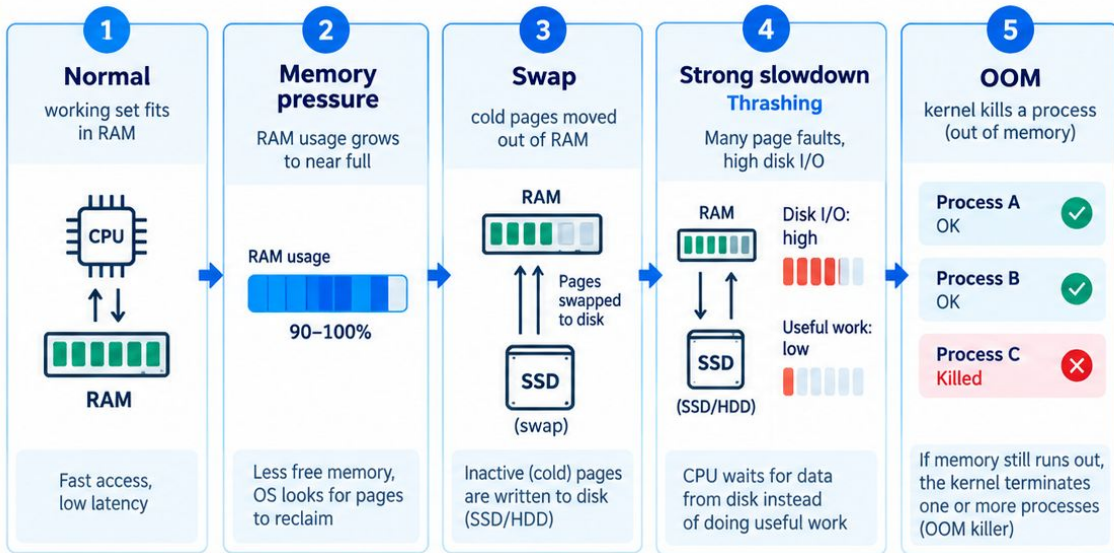
Remote RAM is slower than local RAM

Доступ к удалённой памяти заметно медленнее, чем к локальной.



Swap is not extra RAM

Swap находится на накопителе (SSD/HDD) и на порядки медленнее, чем RAM.



Practical consequences (symptoms)



Slow response

Programs and the whole system become sluggish, high latency.



Disk I/O high

High read/write activity on SSD/HDD (e.g. high %util in iostat).



Process killed

Kernel may kill a process (OOM killer).

```
dmesg | tail -n 2  
Out of memory: Killed process 1234  
(python) total-vm: 8123456kB, ...
```

Диски

Дисковые отсеки

hot-swap
замена без выключения



Накопители

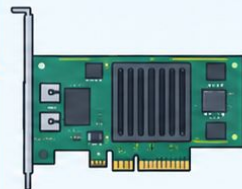
SSD

NVMe

SAS



Контроллер / RAID



Файловая система



Данные и приложения



Файлы



Логи



Базы данных



Образы VM



Резервные копии



Подсистема хранения

- несколько накопителей
- единое логическое хранилище
- скорость + надежность + обслуживание

Мифический RAID

RAID 0 — чередование (striping)



- Данные делятся на полосы и раскладываются по нескольким дискам.
- Если один диск выходит из строя, теряется часть каждой полосы → массив недоступен.



Скорость высокая



Отказоустойчивость нет



Емкость эффективная

RAID 1 — зеркалирование (mirroring)



- Каждый блок хранится в двух копиях.
- Если один диск выходит из строя, данные остаются на второй копии.



Скорость средняя

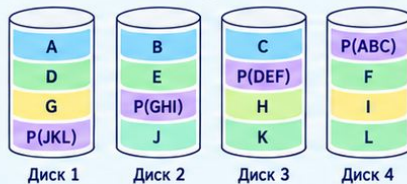


Отказоустойчивость высокая



Емкость низкая

RAID 5 — чередование с контролем четности



- Parity позволяет восстановить один потерянный блок в каждой полосе по оставшимся данным.
- Поэтому массив переживает отказ одного диска.



Скорость средняя



Отказоустойчивость средняя



Емкость хорошая

RAID 10 — зеркалирование + чередование



- Сначала зеркалирование, затем распределение нагрузки между парами.
- Можно пережить отказ одного диска в паре, пока жива вторая копия.



Скорость высокая



Отказоустойчивость высокая



Емкость средняя

Hot-swap и rebuild (на примере RAID 5)

1 Нормальный режим

Все диски в массиве, система работает.



2 Отказ диска

Один из дисков вышел из строя. Массив работает в degraded mode. Недостающие блоки временно восстанавливаются по parity и оставшимся данным.



Массив работает в degraded mode.

3 Hot-swap замена

Вынимаем неисправный диск и устанавливаем новый. Сервер не выключаем.



Вынимаем неисправный диск

Устанавливаем новый диск (сервер не выключаем)

4 Rebuild (восстановление)

Контроллер пересобирает недостающие данные на новый диск по данным с остальных дисков и parity.



Массив снова в нормальном режиме.

RAID 6 тоже существует: похож на RAID 5, но выдерживает отказ 2 дисков.

Сеть

Сервер / NIC



- Встроенный или отдельная PCIe-карта
- Несколько сетевых интерфейсов
- 1 / 10 / 25 / 100 GbE

Типы подключений

RJ-45 Медный Ethernet



До 1 GbE
(1000 Мбит/с)

Витая пара
(4 пары = 8 жил)



SFP / SFP+ (модульный формат)



Один и тот же слот —
разные типы модулей:

Медный
модуль
(до 1/10 GbE)

Оптический
модуль
(10/25/100 GbE)

Оптоволокно

Высокие скорости
и большие расстояния



10 / 25 / 40 / 100 GbE

Оптический кабель
(например, LC)



Кабель и среда



Витая пара
(Cat5e / Cat6 / Cat6a)
до 1000 Мбит/с



SFP (медь)
до 1 / 10 GbE
(зависит от модуля)



Оптоволокно
10 / 25 / 40 / 100 GbE
(большие расстояния)

Коммутатор (Switch)

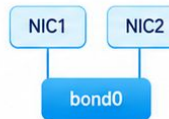


- Принимает разные типы подключений (RJ-45, SFP и др.)
- Объединяет устройства в сеть
- Работает на требуемых скоростях (1 / 10 / 25 / 100 GbE)

Несколько NIC и bonding (кратко)



- Несколько сетевых интерфейсов
- Для разделения трафика, резервирования
- Объединение (bonding / teaming)



Сеть — ограниченный ресурс



Bandwidth

Ограниченная
пропускная способность

1 / 10 / 25 / 100 GbE



Latency

Задержки
при передаче

мкс — мс



Packets / Queues

Ограниченное количество
пакетов в очередях

зависит от NIC и нагрузки

Нажали на кнопку



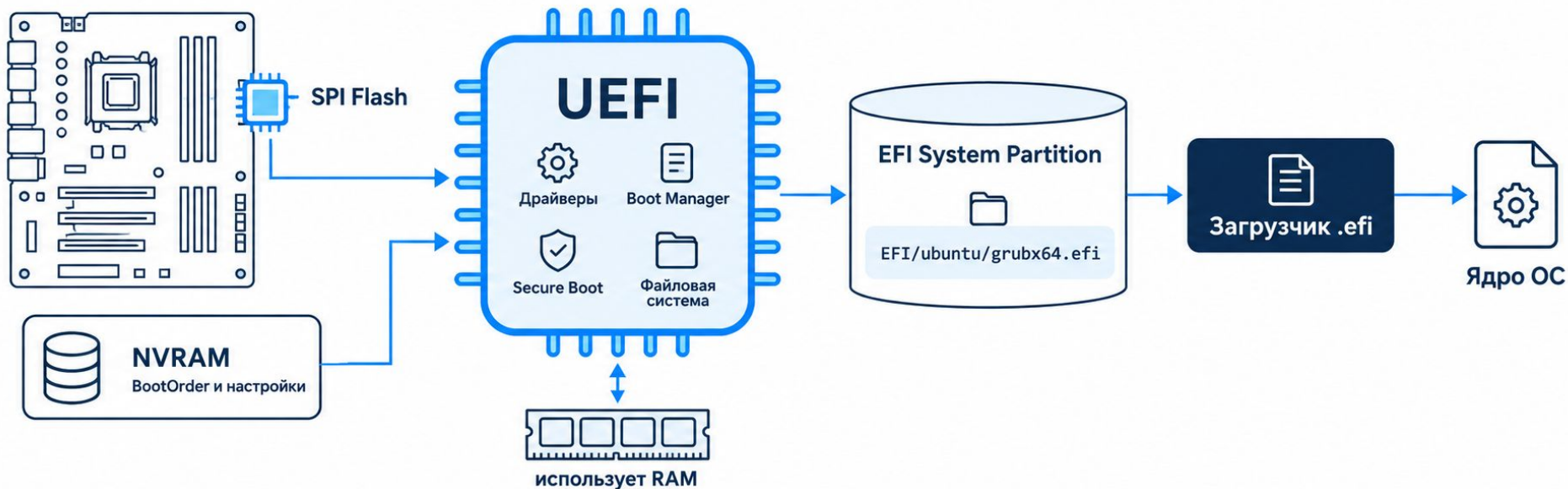
Linux ещё не загружен: нет процессов, сервисов и подключённой файловой системы

BIOS



BIOS подготавливает компьютер и запускает код из MBR

UEFI



UEFI

EFI-раздел

.efi-загрузчик

Ядро ОС

ExitBootServices()

UEFI находит .efi-файл, запускает загрузчик и передаёт управление ядру

Запуск



Что находится в RAM

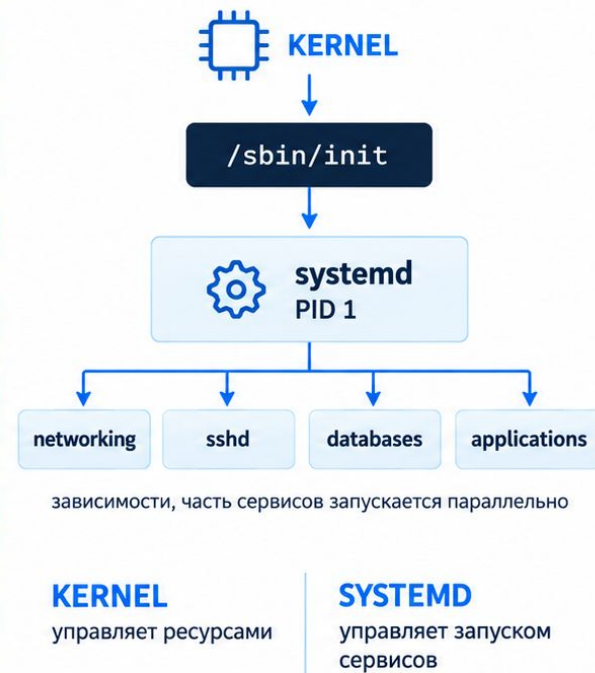
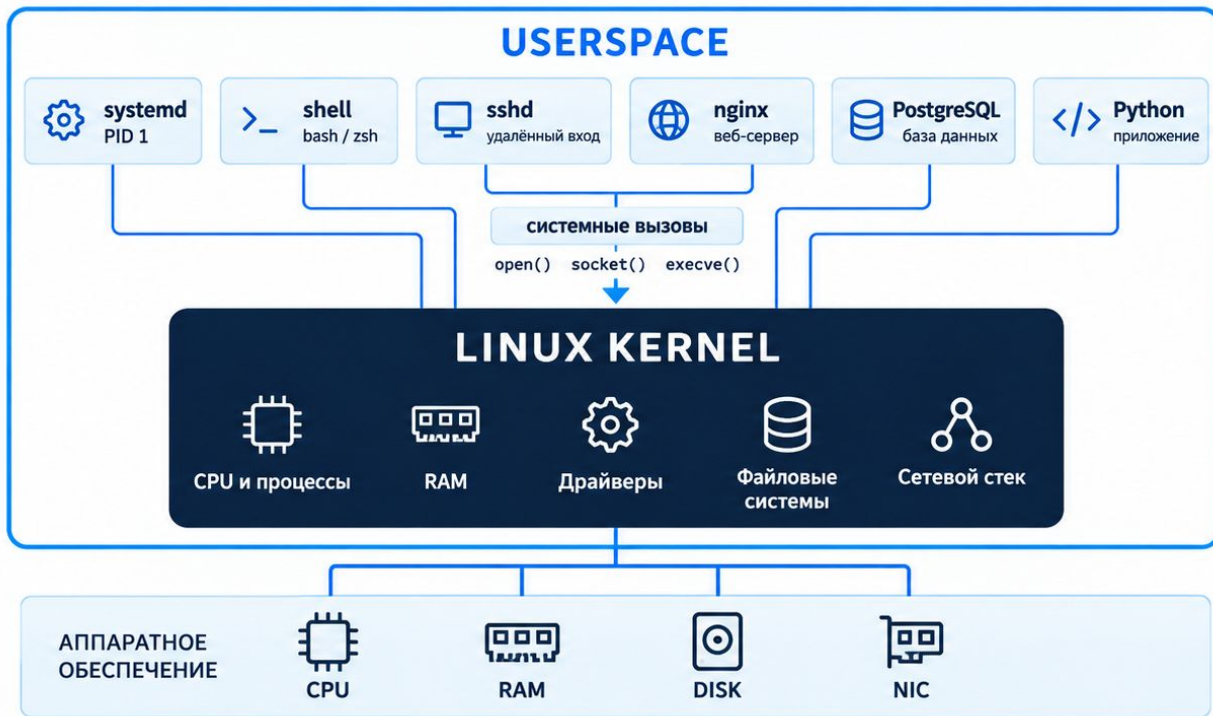
GRUB	vmlinuz	initramfs	параметры	передача адресов и параметров	распакованное ядро	таблицы памяти	initramfs
------	---------	-----------	-----------	-------------------------------	--------------------	----------------	-----------

Кто управляет машиной



GRUB загружает образ ядра. Kernel распаковывается, инициализирует оборудование и запускает первый пользовательский процесс.

Linux



Kernel управляет аппаратными ресурсами. Userspace содержит сервисы и приложения.

Кто такие сервисы

myapp.service

[Unit]

Description=My Python application
Wants=network-online.target
After=network-online.target

[Unit]

зависимости и порядок

[Service]

User=artem
WorkingDirectory=/opt/myapp
EnvironmentFile=/etc/myapp/app.env
ExecStart=/usr/bin/python3 /opt/myapp/app.py
Restart=on-failure

[Service]

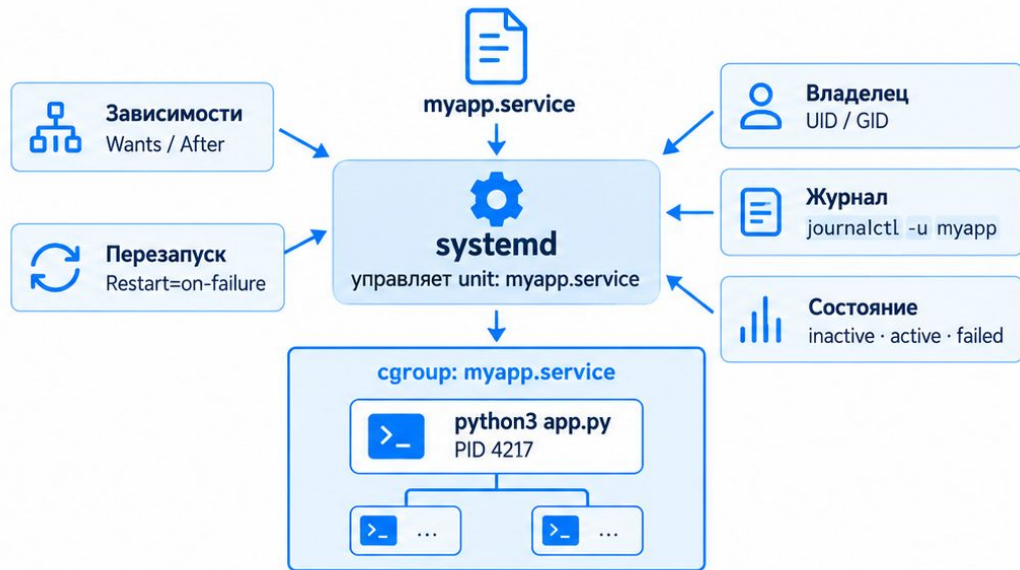
запуск и контроль
процесса

[Install]

WantedBy=multi-user.target

[Install]

автозапуск



```
$ systemctl start myapp
$ systemctl status myapp
$ systemctl enable myapp
```

start запускает сейчас · enable включает автозапуск

Процесс выполняет программу. Service unit задаёт, как systemd запускает, контролирует и перезапускает процесс.

В чем сила, root ?



Текущая shell не становится root. Sudo запускает новый дочерний процесс.

Обычный запуск

```
artem$ python3 app.py
```

UID	1000
USER	artem
HOME	/home/artem
PATH	содержит .venv/bin
ENV	переменные пользователя
Файлы	artem:artem
Доступ	права пользователя



другая идентичность
и другое окружение

Запуск через sudo

```
artem$ sudo python3 app.py
```

UID	0
USER	root
HOME	зависит от политики sudo
PATH	secure_path из sudoers
ENV	часть переменных удалена
Файлы	root:root
Доступ	административные права



Другой Python

```
.venv/bin/python3 ≠ /usr/bin/python3
```

ModuleNotFoundError



Другой владелец

```
build/result.json root:root
```

Permission denied при обычном запуске



Системный масштаб ошибки

```
/etc /usr services users
```

ошибка или уязвимость затрагивает всю машину

Root — пользователь с UID 0. Sudo запускает новый процесс, а не усиливает текущий.

Пути и переменные

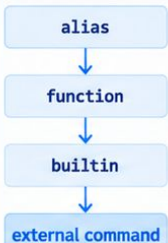
```
$ APP_MODE=production python app.py --port 8080
```

переменная для запуска

имя команды

аргументы

1 Разрешение команды



PATH

```
1 /home/artem/project/.venv/bin ✓ FOUND
2 /home/artem/.local/bin
3 /usr/local/bin
4 /usr/bin
5 /bin
```

Результат:

/home/artem/project/.venv/bin/python



- первое совпадение побеждает
- ./script.sh содержит / → PATH не используется

2 execve()

```
execve(
    "/home/artem/project/.venv/bin/python", path — executable
    ["python", "app.py", "--port", "8080"], argv — аргументы
    ["HOME=/home/artem",
     "PATH=...",
     "APP_MODE=production"] envp — окружение
)
```



kernel получает уже готовый путь

3 Новый процесс



python app.py

PID 4217

executable /home/artem/project/.venv/bin/python

argv app.py --port 8080

environment HOME=/home/artem
APP_MODE=production
PORT=8080



- ENV копируется при создании процесса
- изменения shell не меняют уже работающий процесс

```
type -a python
```

все варианты

```
command -v python
```

выбранный executable

```
printenv APP_MODE
```

значение переменной

```
env | sort
```

полное окружение



APP_MODE=development

debug=on
database=local
port=8080

PID 4218

← один executable
два окружения
два процесса →



APP_MODE=production

debug=off
database=production
port=80

PID 4219

PATH выбирает executable. argv задаёт аргументы. envp формирует конфигурацию нового процесса.

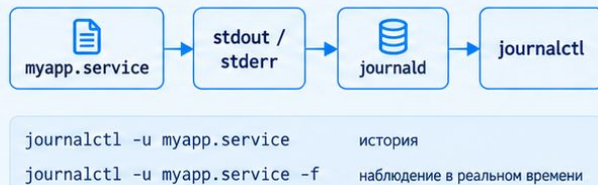
Логи



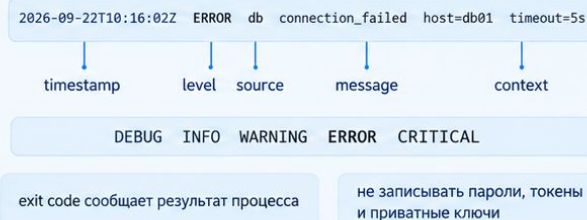
Примеры использования в shell

<code>python app.py < input.txt</code>	stdin из файла
<code>python app.py > output.log</code>	stdout в файл
<code>python app.py 2> errors.log</code>	stderr в файл
<code>python app.py > app.log 2>&1</code>	оба потока в один файл
<code>python app.py & grep ERROR</code>	оба потока в pipe

Логи сервиса в systemd



Структура одной записи лога



Процесс пишет в stdout и stderr. Shell, systemd или контейнерная среда определяют, куда попадут сообщения.

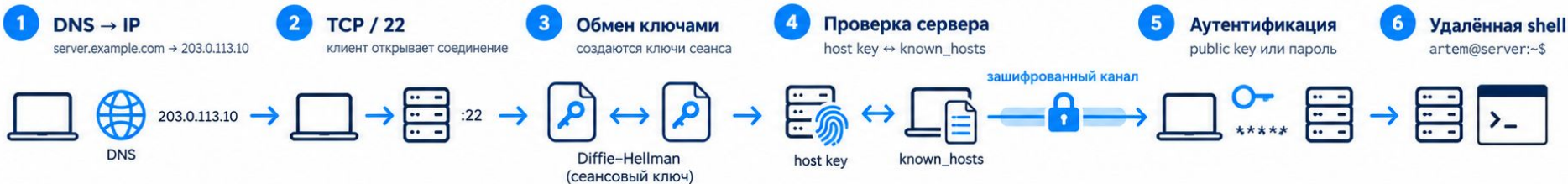
SSH

```
$ ssh artem@server.example.com
```

пользователь

сервер

порт 22 по умолчанию



HOST KEY

подтверждает сервер

сервер: /etc/ssh/ssh_host_ed25519_key

клиент: ~/.ssh/known_hosts

изменился ключ → сначала проверить причину



USER KEY

подтверждает пользователя

клиент: ~/.ssh/id_ed25519 – private key

сервер: ~/.ssh/authorized_keys – public key

private key не передаётся по сети

```
ssh-keygen -t ed25519
```

```
ssh-copy-id artem@server.example.com
```

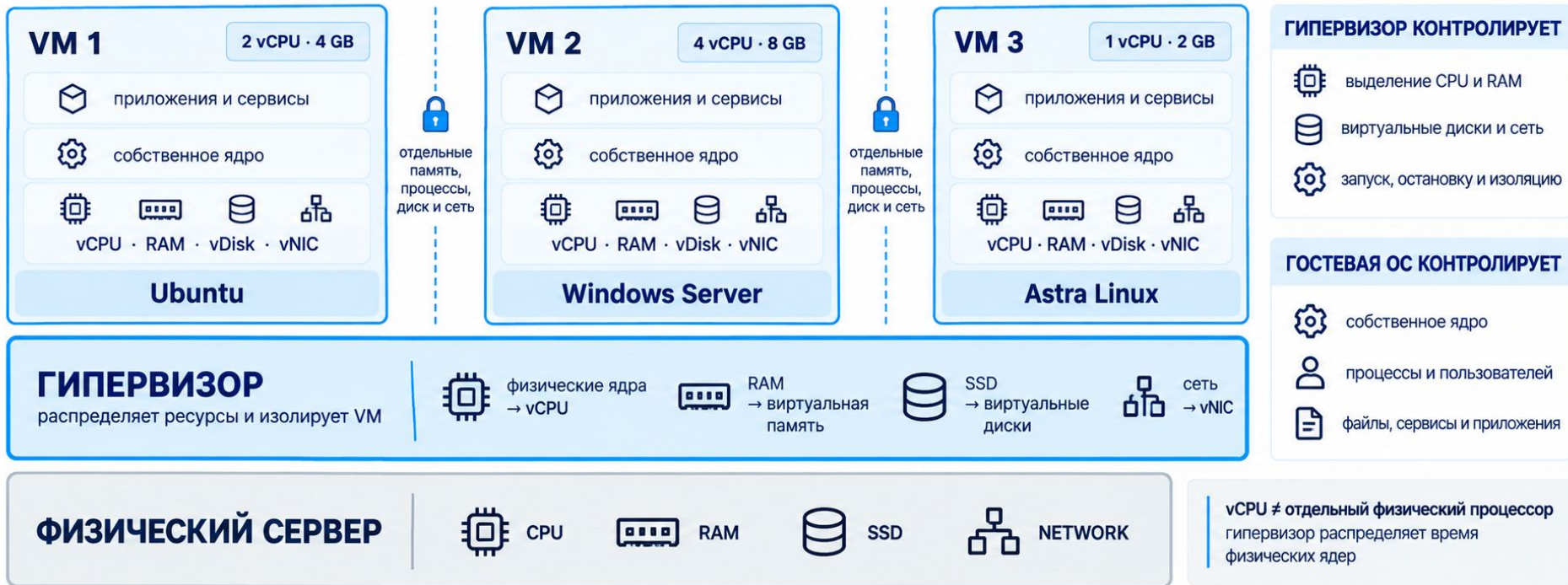
Host key подтверждает сервер. User key подтверждает пользователя. SSH защищает весь последующий обмен.

22 . . .



Через одно SSH-соединение передаются команды, файлы и сетевой трафик.

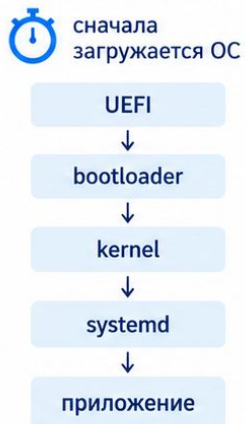
Гипервизоры



VM — виртуальный компьютер с собственным ядром. Гипервизор связывает его с ресурсами физического сервера.

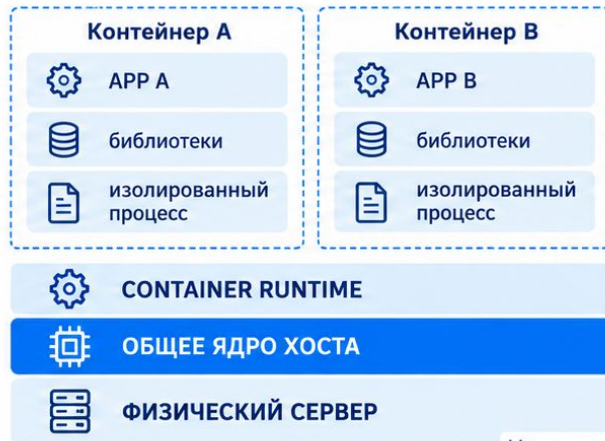
VM vs Контейнер

ВИРТУАЛЬНАЯ МАШИНА



VS

КОНТЕЙНЕР



Linux-контейнеру нужен Linux kernel

собственный	KERNEL	общий с хостом
загрузка ОС	ЗАПУСК	запуск процесса
отдельное ядро	ИЗОЛЯЦИЯ	изоляция процессов
больше накладных расходов	РЕСУРСЫ	меньше накладных расходов

VM изолирует целую операционную систему. Контейнер изолирует приложение и его процессы.

